

Steps for building an application for searching resume details by HR

Mongodb Set up :

1. Open Mongodb URL
2. Click on sign up
3. Give a new gmail ID and password
4. After logging in click on Free trail in clusters
5. Click on create deployment
6. Copy the username and password displayed. Download the .env file. Username and password will be in .env file also
7. Create new database, collection and Vector search index. Copy and paste the Json file given for Vector Search Index and BM25 search index in Mongodb.
8. For BM25 create a separate collection and vector index
Note :
9. For Vector Index select 'Vector Search'. Create your own option. Select the folder displayed
10. For BM25, Select Atlas search
11. Go to database. Click on edit. Click on edit password. Click on autogenerate password. Copy the password and paste the password in Mongodb URI.
12. Go to Clusters. Click on Connect. Click on Drivers. Mongodb URI will be there. Copy it in notepad and paste the password.
13. Disable SRV and copy the Mongodb URI

API Keys :

Create Groq API key in Console.groq url

Create Mistral API key

General Instructions :

1. Check the Architecture document(01_Resume_Ingestion_Phasewise implementation 1.md) in Vs code
2. Under each phase the purpose of each phase and expected output in Postman will be given. So once response is received in Postman check if it matches with the given output in architecture md file in VS code.

3. Then kill the terminal. Go to Terminal-New Terminal. Type 'npm run dev'. We are making changes to the code through API. So we need to rerun the server and next proceed with next phase

4. Then type in copilot within VS code 'Approved, Please proceed with next phase'

Steps to follow :

1. Create an empty folder with name 'Resume_Rag'

2. Place the architecture file inside this folder. This file will be generated by giving ICEPOT prompt.

3. Open VS code. File-New Window. File-open folder. Give the below path

E:\Vidhya_Assignment link\August 30th\Resume_Rag

4. In Copilot within VScode give the below prompt

Prompt :

Refer my Ingestion pipeline architecture document and do implement phase by phase.

Once one phase is completed ,give me Request body and expected response to test in postman .once i tested .after my approval go

for phase 2.same to be followed for rest of the phases also

[STRICT]DONT GO FOR NEXT PHASE UNTIL I APPROVE

[STRICT]DO NOT HALLUCINATE .READ THE ARCHITECTURE CAREFULLY AND START IMPLEMENTATION

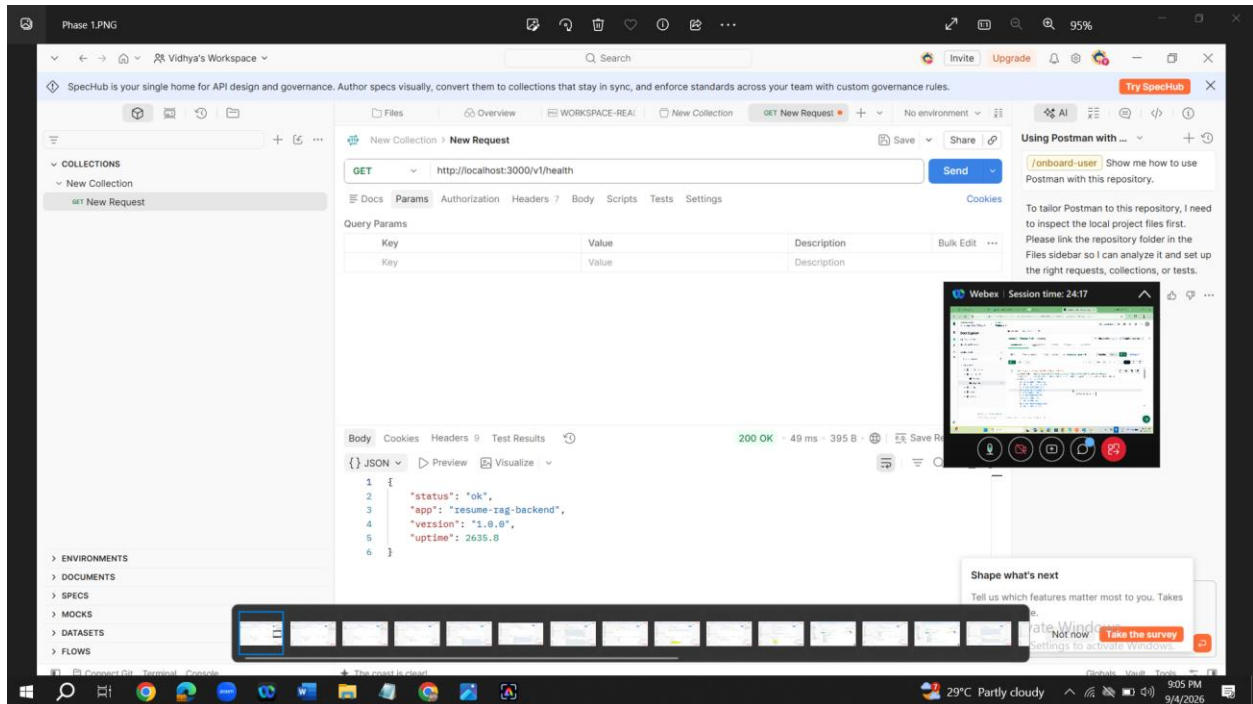
5.Click on New-Terminal

6. Type 'npm run dev'

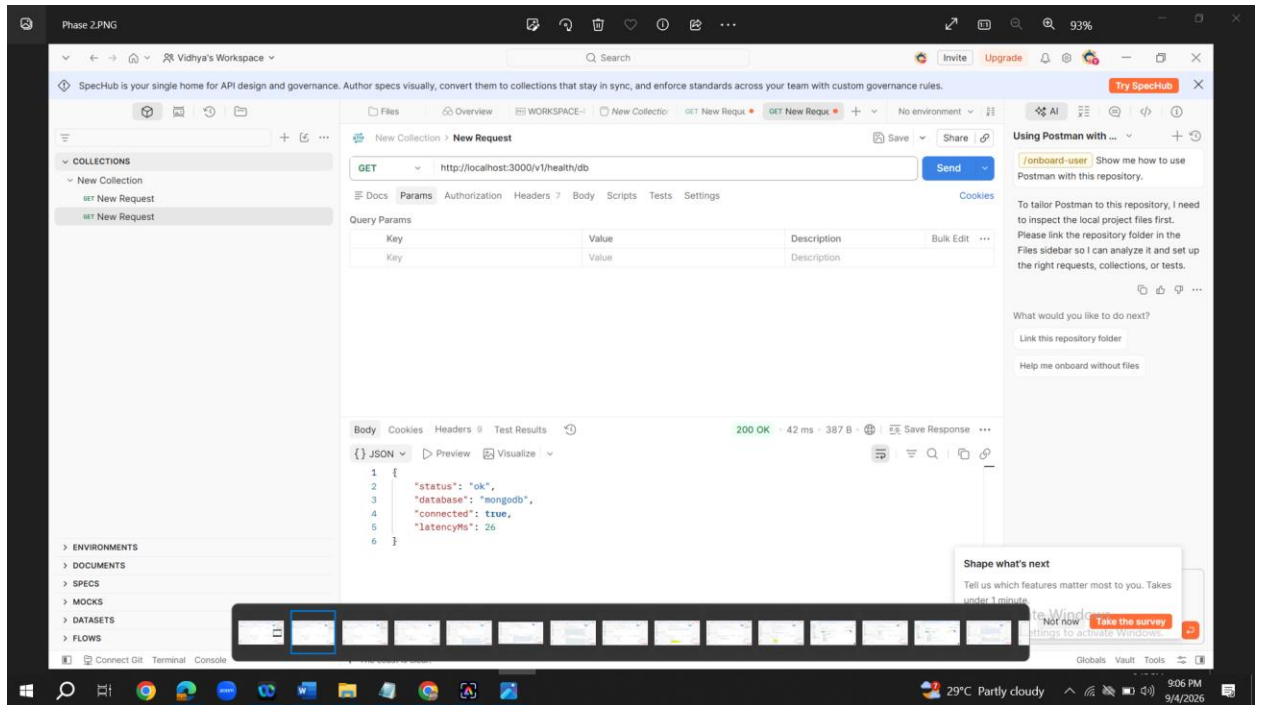
7. In the result of the prompt given, URL will be given. Copy the URL.

8. Open Postman

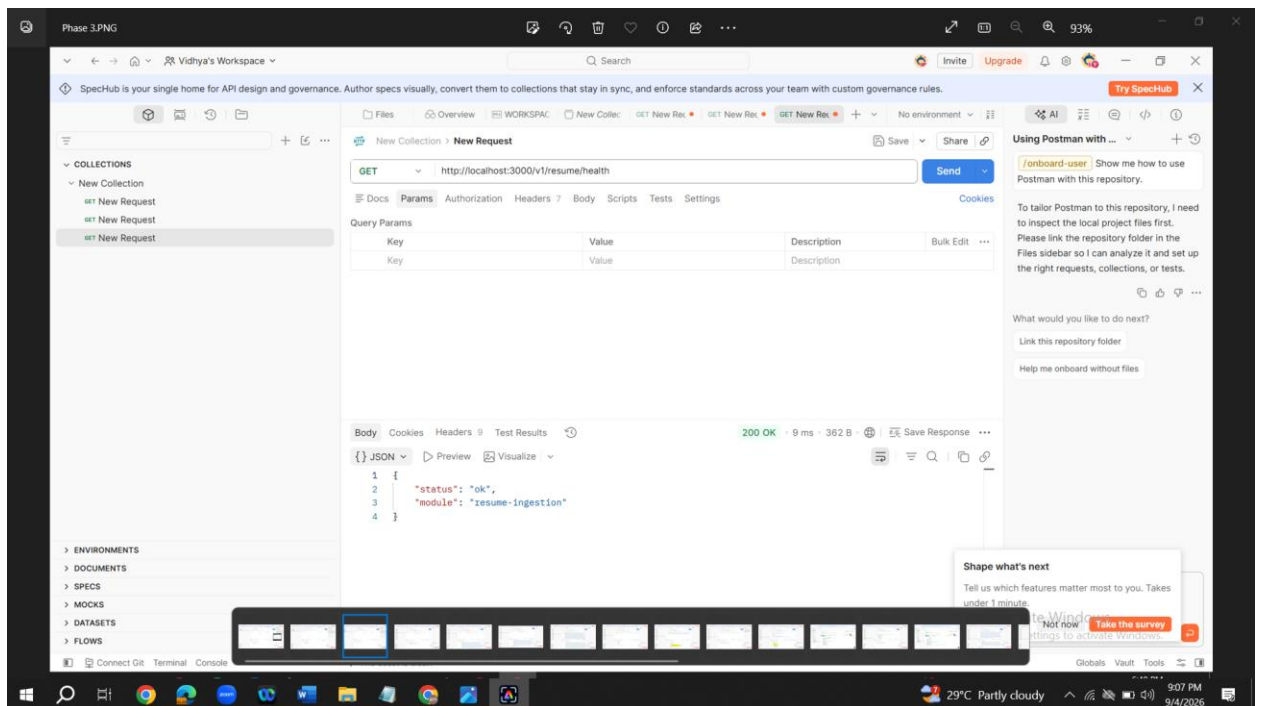
9. Click on '+' icon and select collections. Rename the collection to Resume_Rag
10. Under the collection, click on 'New Request'
11. Near 'GET', paste the URL copied in Copilot results of Phase 1
12. It should be successful returning '200' ok



13. In copilot within VS code type 'Approved, please proceed with Phase 2'



14. Follow the same steps from step 7 to step 13

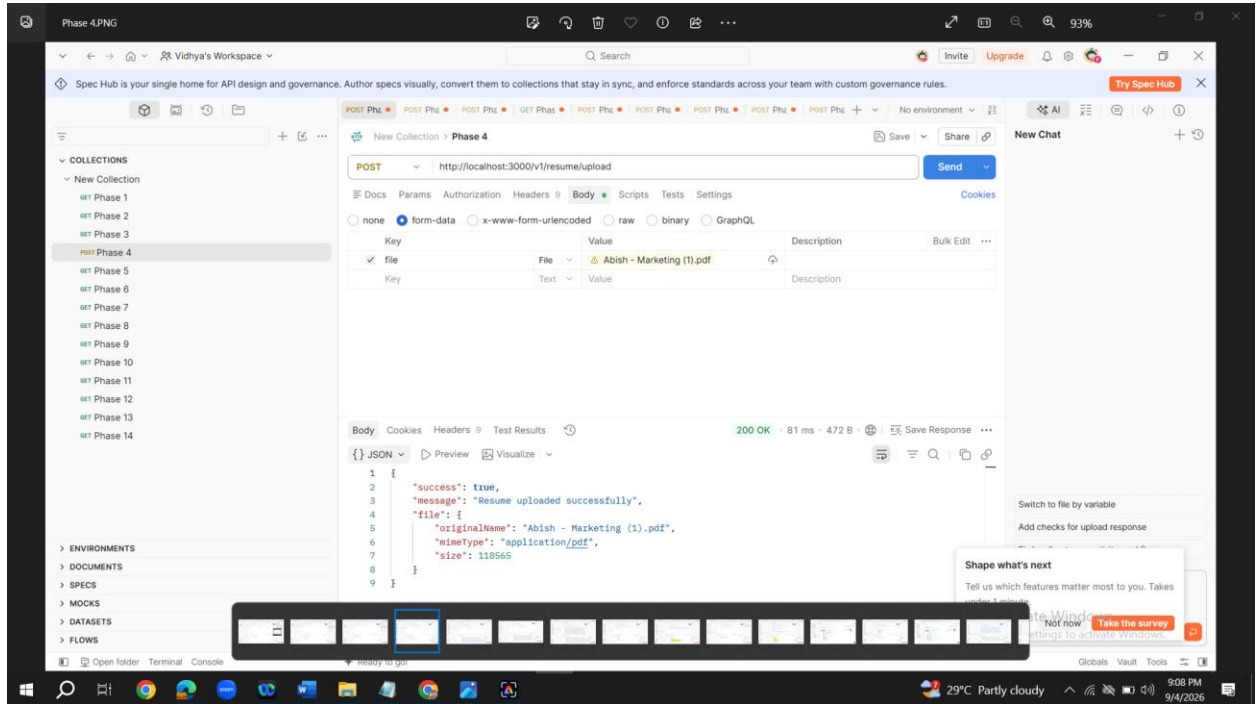


15. For phase 4, in Postman change it to 'POST' and give the URL

16. Navigate to Body tab in Postman and select 'Form-data'. In Key type 'file'. Select file type as 'File'. In value upload the resume given. It is a PDF file. Click on Send

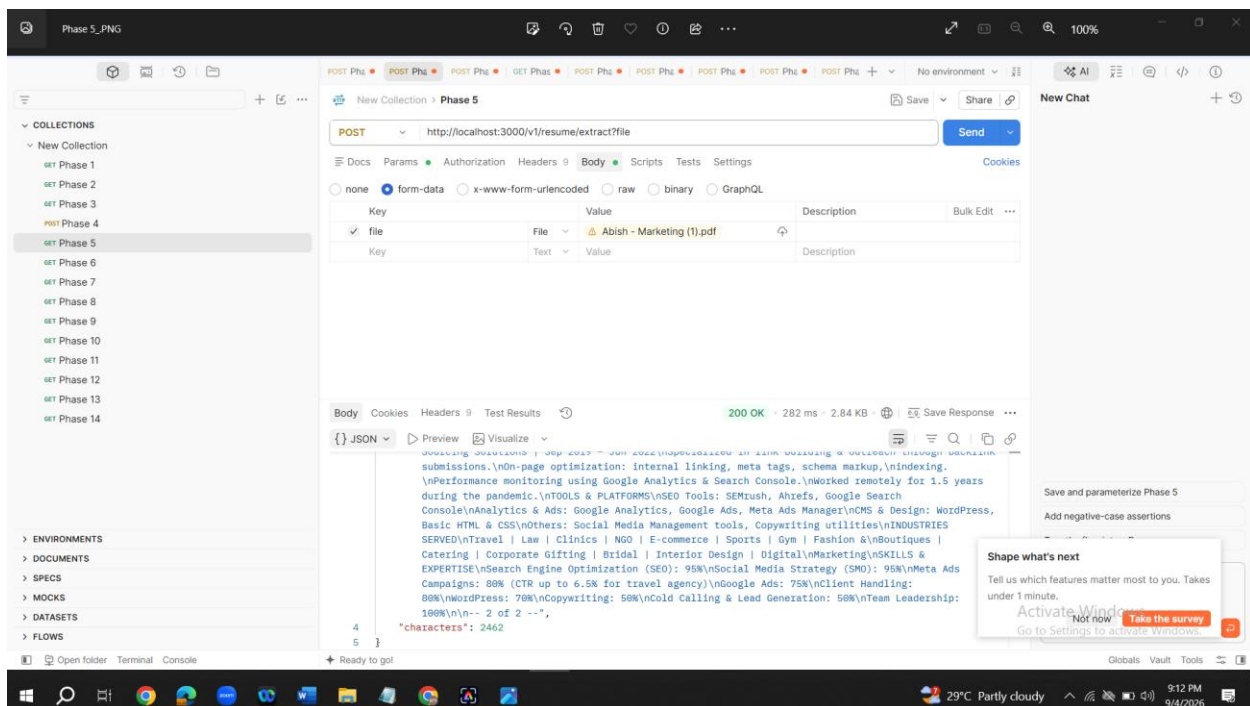
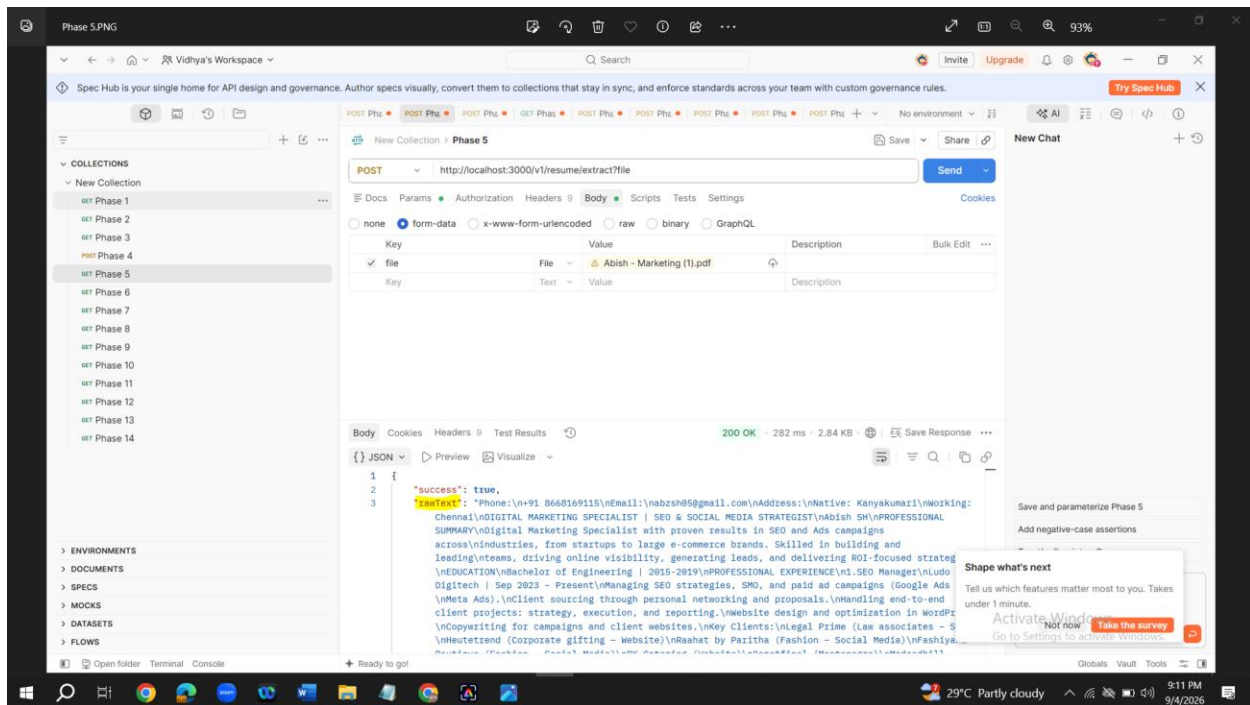
Phase 4 : <http://localhost:3000/v1/resume/upload>

17. Success message with return code '200' ok should be displayed



18. For Phase 5, in Postman change it to 'POST' and give the URL

16. Navigate to Body tab in Postman and select 'Form-date'. In Key type 'file'. Select file type as 'File'. In value upload the resume given. It is a PDF file. Click on Send. In the results copy the json content inside "rawText" upto 2 of 2--". Dont copy the 'characters : 2462'.



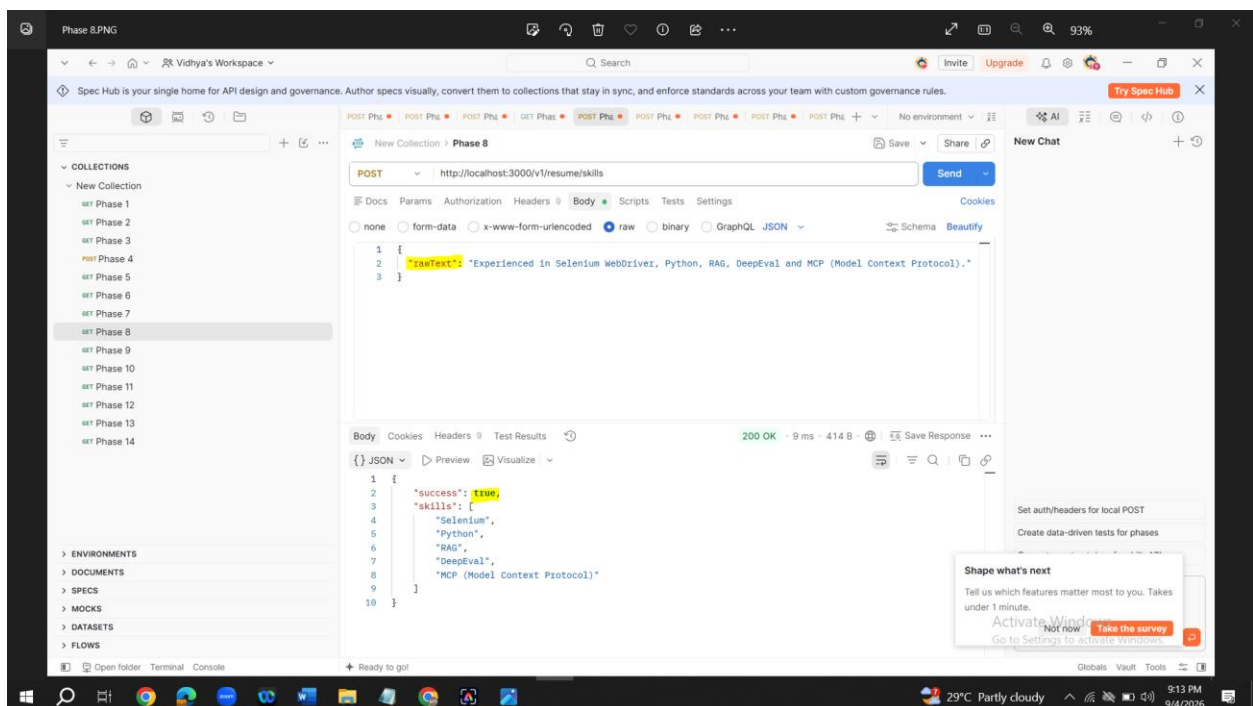
17. In Phase 6, Select POST Method in Postman and copy the Url given by Copilet and paste it in postman. Navigate to Body tab. Select raw radio button. Paste the json copied in Phase 5 inside "rawText". Format will be given by copilet. So just paste the json in the given format by copilet.

```
{
```

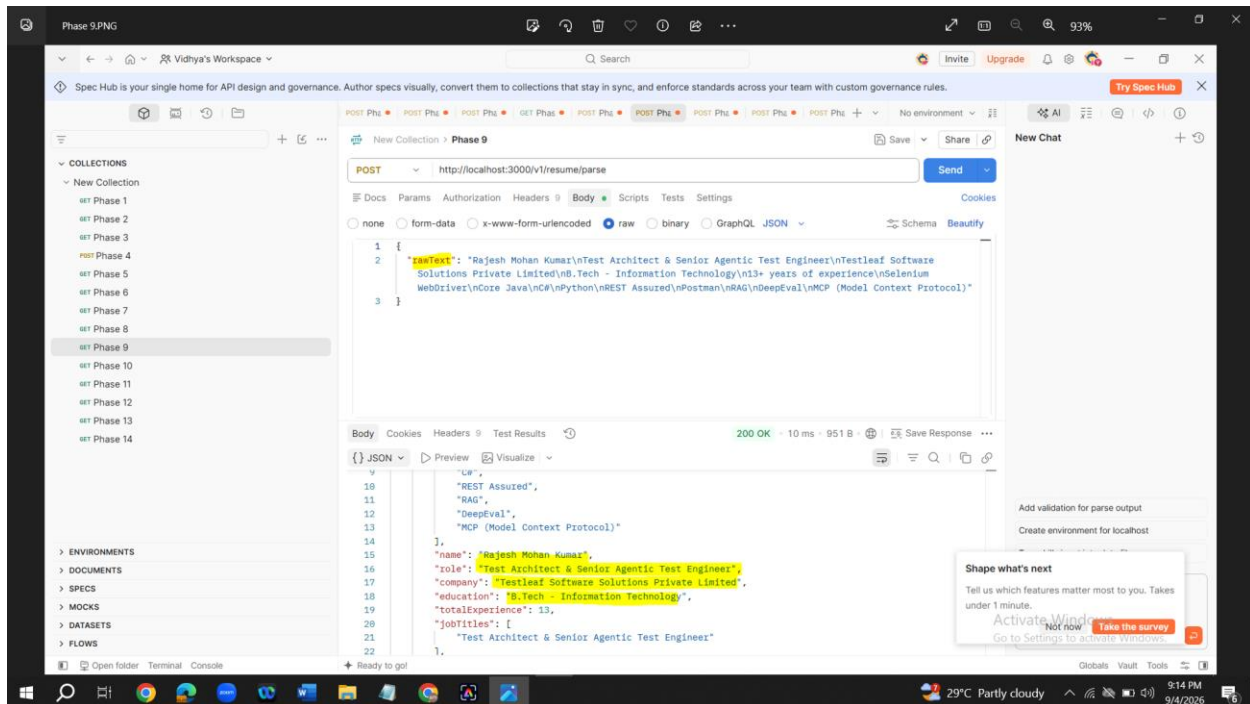
"rawText" : "Paste Json"

18. For Phase 7, no API testing is needed in postman

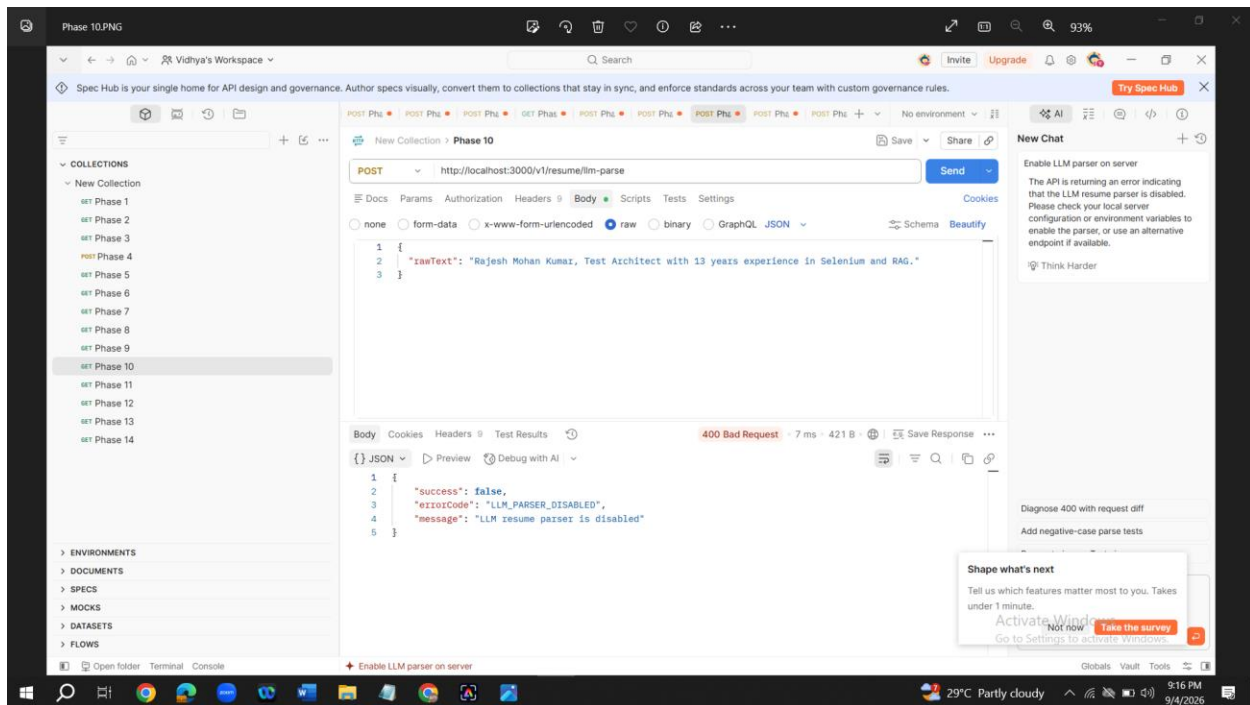
19. For Phase 8, just copy the contents give by copilet in Postman following the instructions. Method-Post. Copy URL. Navigate to Body tab. Select 'raw' radio button. Copy and paste the contents given by Copilet.



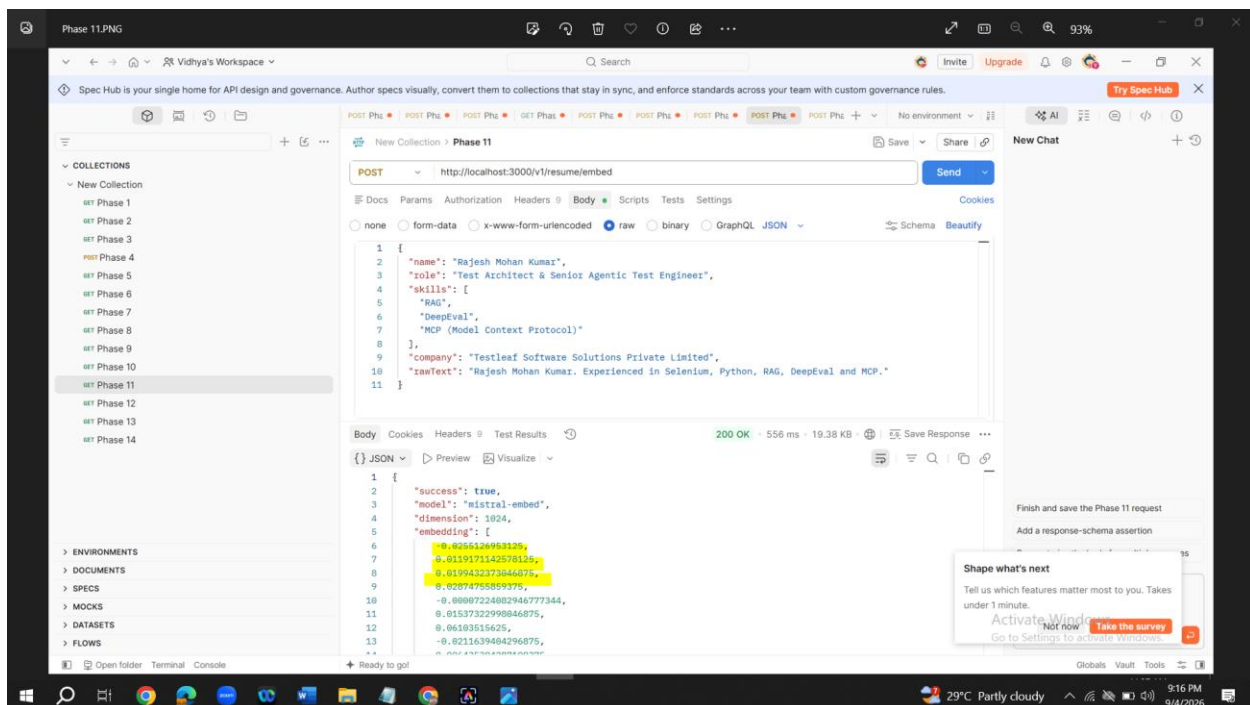
20. For Phase 9, just copy the contents given by Copilot in Postman following the instructions. Method-Post. Copy URL. Navigate to Body tab. Select 'raw' radio button. Copy and paste the contents given by Copilot.



21. For Phase 10, just copy the contents given by Copilot in Postman following the instructions. Method-Post. Copy URL. Navigate to Body tab. Select 'raw' radio button. Copy and paste the contents given by Copilot.



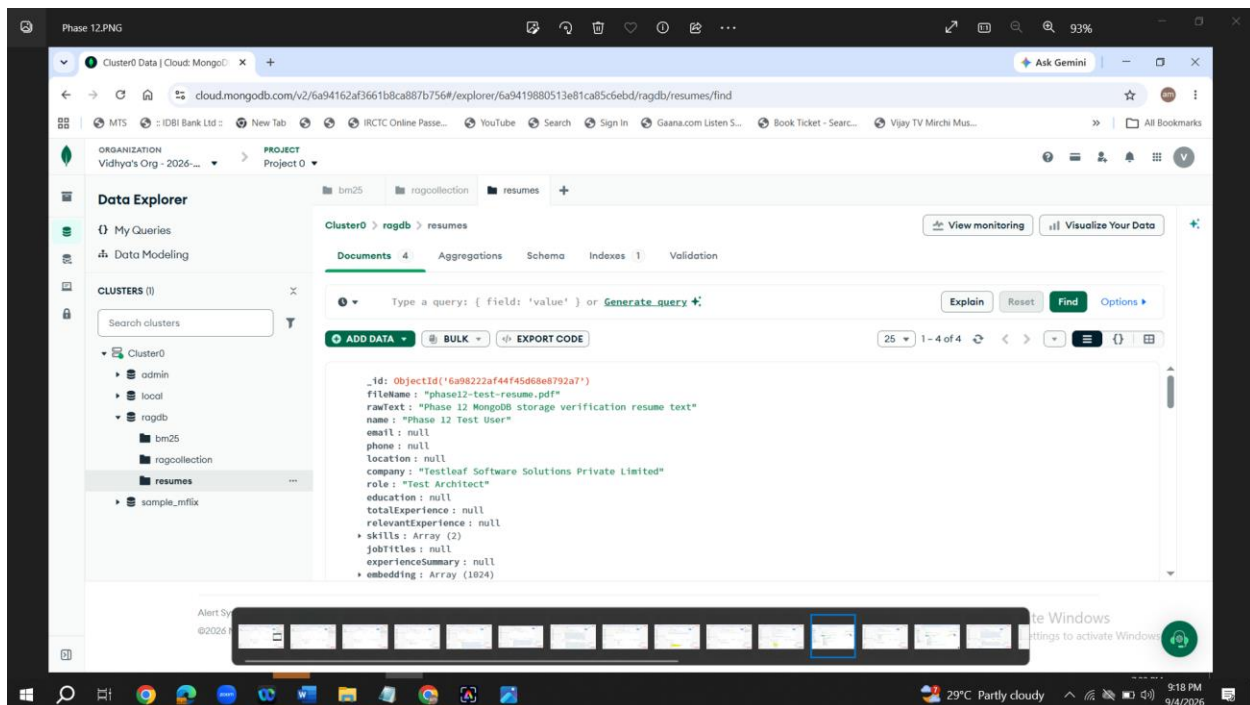
22. For Phase 11, just copy the contents given by Copilot in Postman following the instructions. Method-Post. Copy URL. Navigate to Body tab. Select 'raw' radio button. Copy and paste the contents given by Copilot. In the results a set of embeddings will be listed. Copy the entire list of embeddings displayed within the brackets.

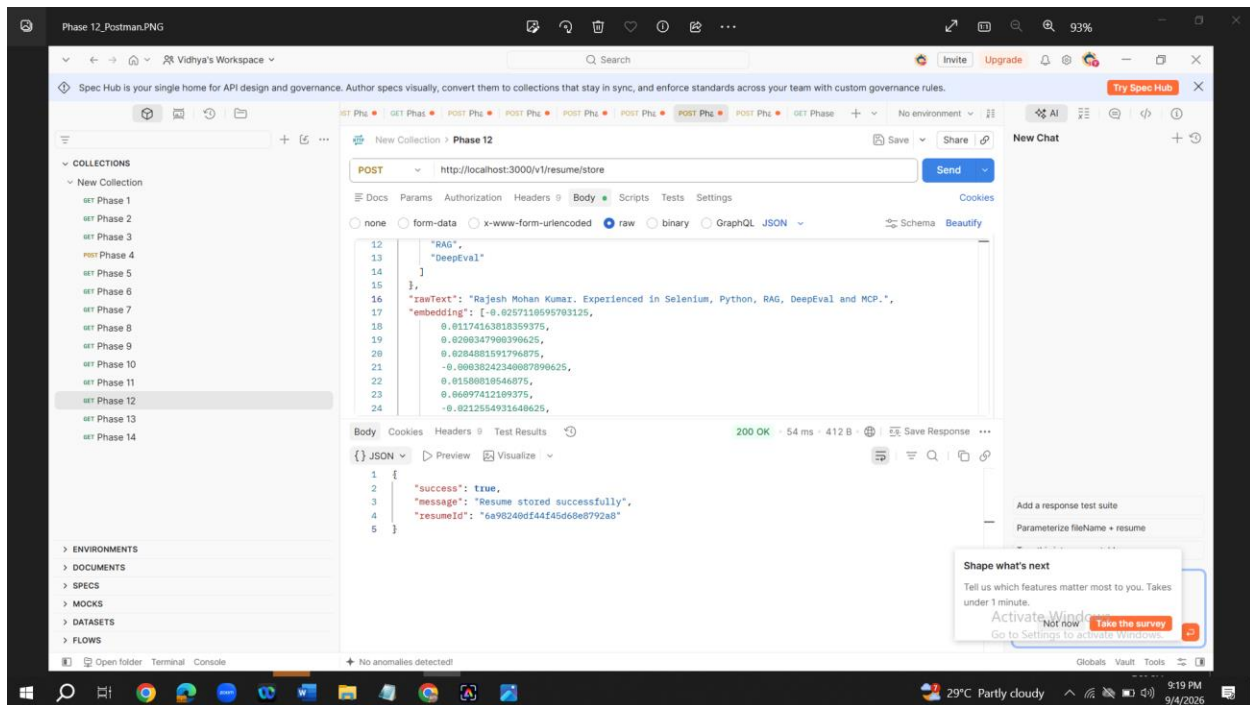


23 . For Phase 12, just copy the contents give by copilet in Postman following the instructions. Method-Post. Copy URL. Navigate to Body tab. Select 'raw' radio button. Copy and paste the contents given by Copilet. For rawText copilet has given the below output(last 2 lines from copilet). Instead of these last 2 lines replace it. Paste the entire list of embeddings in within bracket for second line (embedding). For first line copy the contents in rawText given in Phase 11.

"rawText": "Rajesh Mohan Kumar resume text",

"embedding": [0.01, 0.02]





We can click on 'Find' in MongoDB and paste the resume ID received from postman response from the above screenshot.

24. Data will be loaded into MongoDB under collection 'resumes'

25. Unzip Resumes folder given and place it inside the workspace

E:\Vidhya_Assignment link\August 30th\Resume_Rag\Resumes

26. For Phase 13, **Prompt** after completing Phase 12 :

Approved, Please proceed with Phase 13 by making the following changes

I am having a list of resumes in the 'Resumes' folder. Read resumes from the 'Resumes' folder from the below path.

E:\Vidhya_Assignment link\August 30th\Resume_Rag\Resumes

Modify the codebase in a way that the resumes should be processed batch by batch and configure the workflow to process resumes in batches of 5 or 10 files per run. I want to

implement a batch ingestion workflow for this set of resumes. Validate the ingestion process by testing multiple batches and verifying that all resumes are successfully ingested.

Confirm me the path before processing

27. Give the below prompt to validate output for Phase 13. You can mention any resume name from the list in the **prompt**

To validate output of Phase 13

I need to validate the resume with name 'Arishraj Resume (1)' through API. Could you please tell me how to test it and what request body I need to give in Postman

I want to implement a batch ingestion workflow for this set of resumes. Configure the workflow to process resumes in batches of 5 or 10 files per run. Validate the ingestion process by testing multiple batches and verifying that all resumes are successfully ingested.

just copy the contents given by Copilot in Postman following the instructions.

a. Extract resume text

Postman:

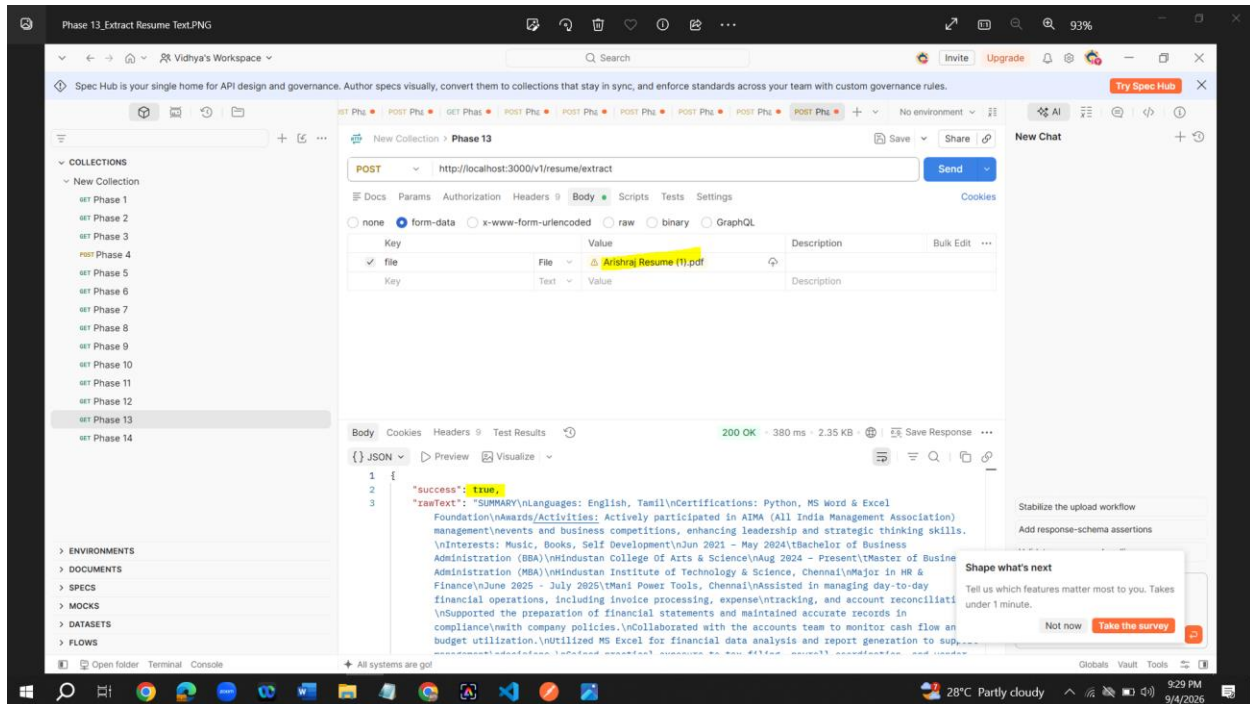
- Method: POST
- URL: `http://localhost:3000/v1/resume/extract`
- Body: form-data
- Key: file
- Type: File
- Value: select Arishraj Resume (1).pdf

No JSON body is required.

Expected Response

```
{  
  "success": true,  
  "rawText": "...",  
  "characters": 1234  
}
```

Copy the returned



b. Clean the text

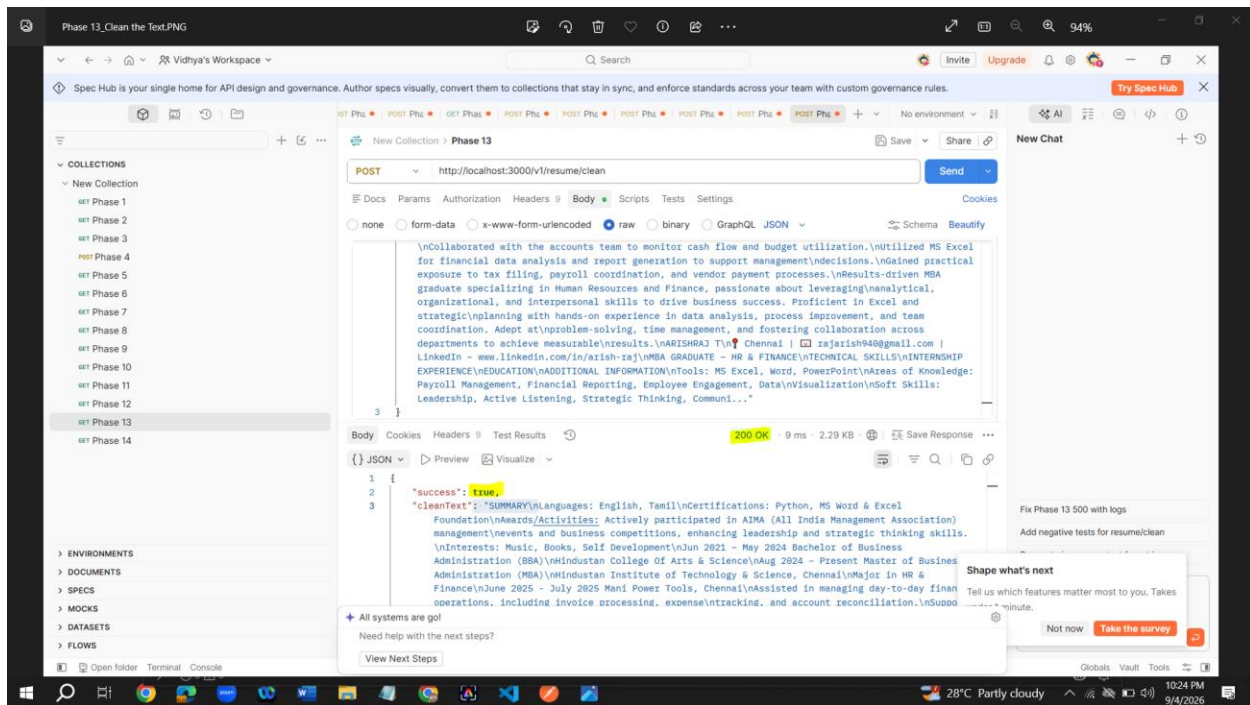
- Method: POST
- URL: http://localhost:3000/v1/resume/clean
- Body: raw
- Type: JSON

{

"rawText": "PASTE_RAW_TEXT_FROM_EXTRACT_RESPONSE"

}

Copy the returned

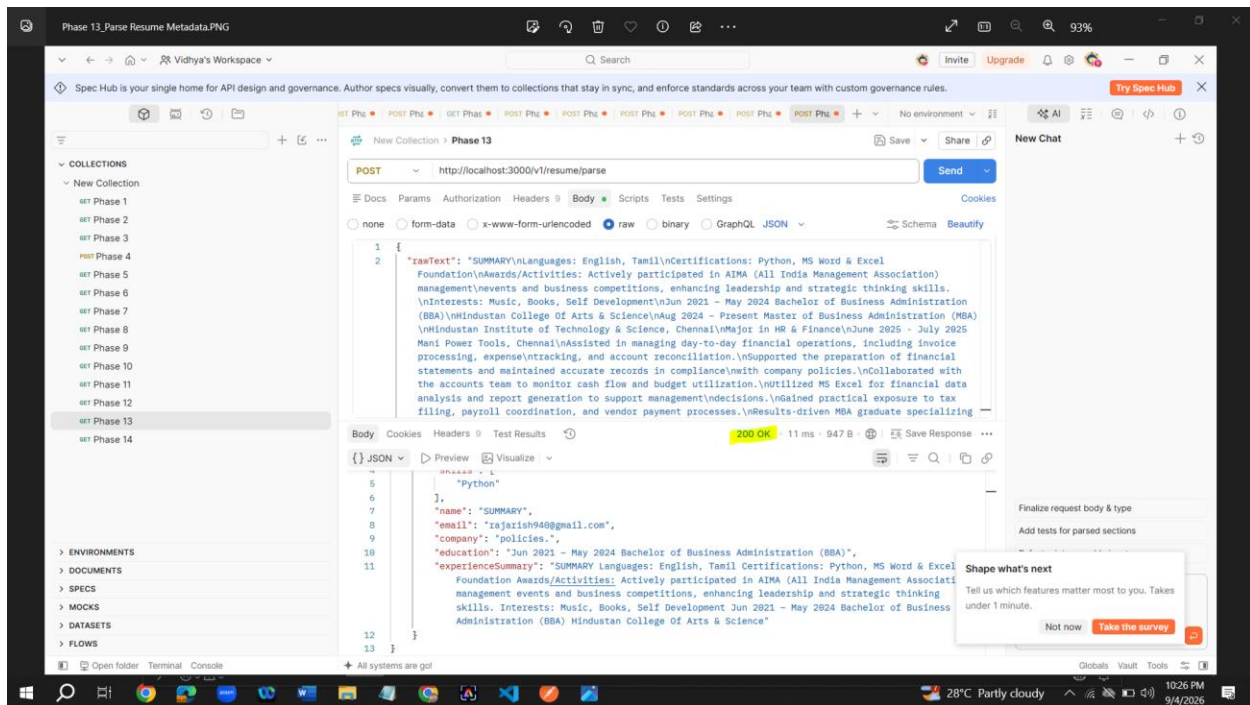


c. Parse resume metadata

- Method: POST
- URL: `http://localhost:3000/v1/resume/parse`
- Body: raw → JSON

```
{  
  
  "rawText": "PASTE_CLEAN_TEXT_HERE"  
  
}
```

Copy the returned resume object.

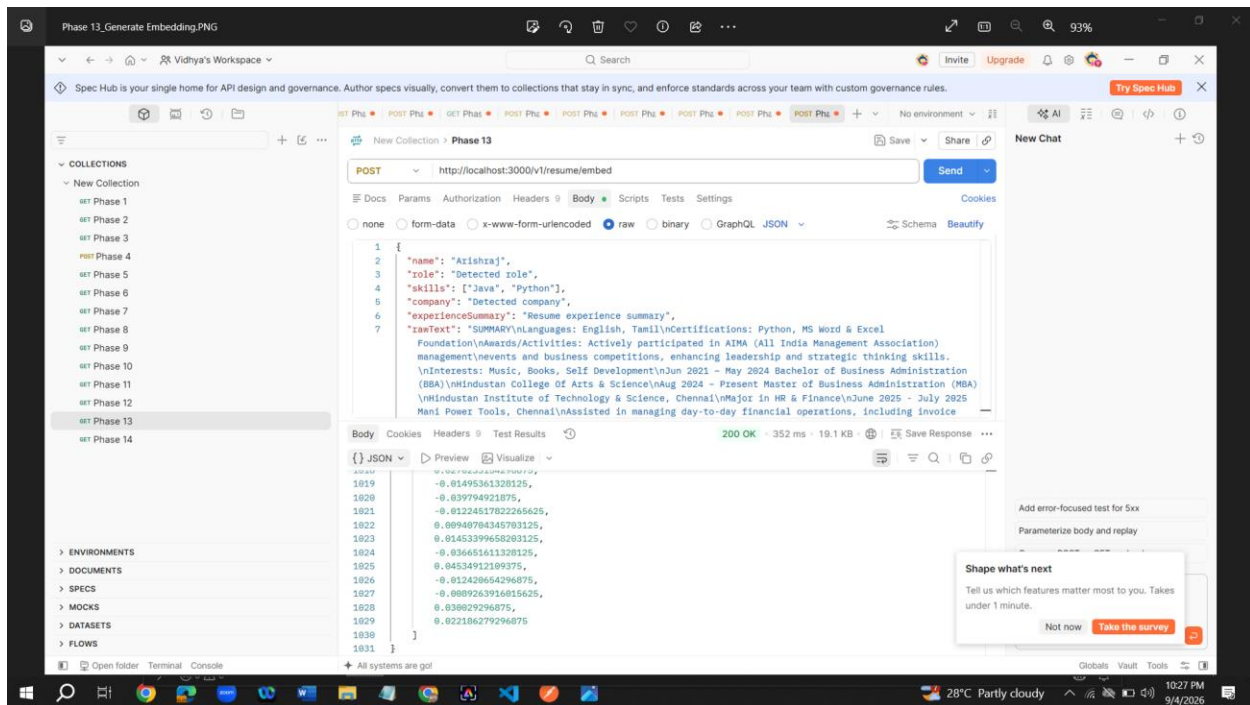


d. Generate embedding

- Method: POST
- URL: `http://localhost:3000/v1/resume/embed`
- Body: raw → JSON

```
{
  "name": "Arishraj",
  "role": "Detected role",
  "skills": ["Java", "Python"],
  "company": "Detected company",
  "experienceSummary": "Resume experience summary",
  "rawText": "PASTE_CLEAN_TEXT_HERE"
}
```

Use the actual values returned by the parse request. Copy the complete embedding array from the response. It must contain the configured number of values, normally 1024.



e. Store the resume

- Method: POST
- URL: `http://localhost:3000/v1/resume/store`
- Body: raw → JSON

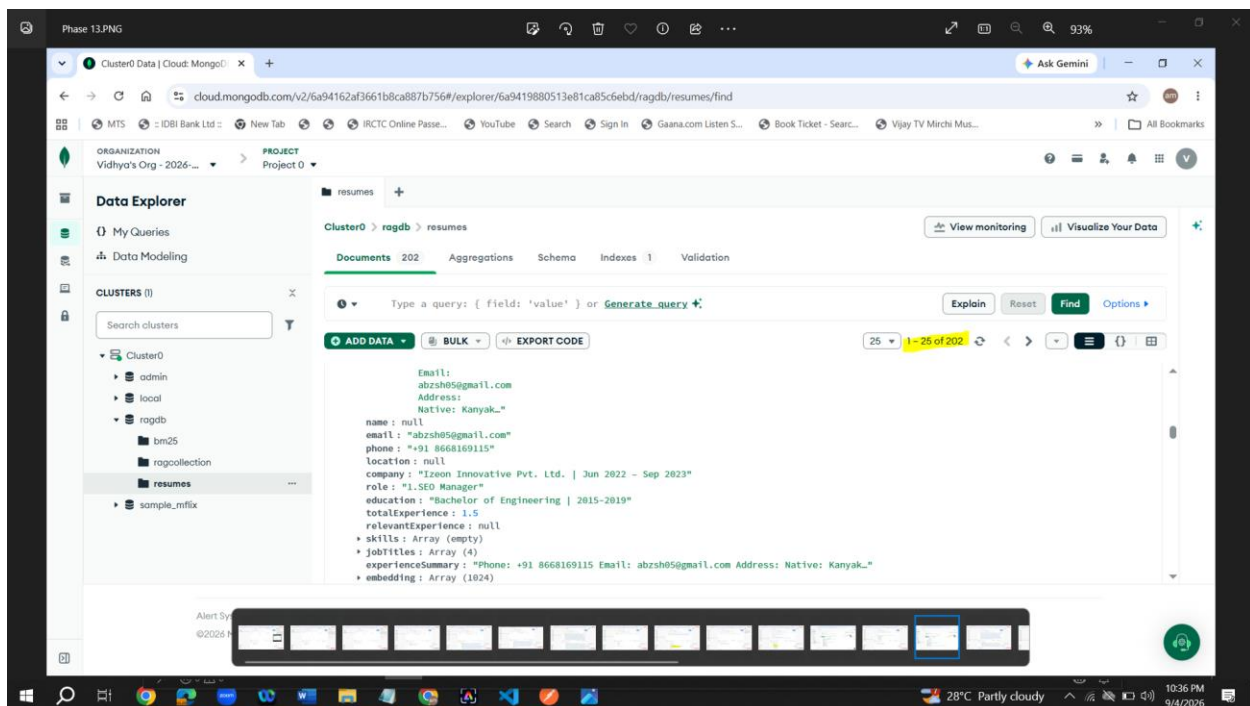
```
{
  "fileName": "Arishraj Resume (1).pdf",
  "resume": {
    "name": "Arishraj",
    "role": "Detected role",
    "skills": ["Java", "Python"],
    "company": "Detected company"
  },
  "rawText": "PASTE_CLEAN_TEXT_HERE",
  "embedding": "PASTE_THE_COMPLETE_EMBEDDING_ARRAY_HERE"
}
```

Expected response:

```
{  
  "success": true,  
  "message": "Resume stored successfully",  
  "resumeld": "..."  
}
```

The upload and extract endpoints currently accept PDF files only. Do not manually type the embedding or use a shortened array; copy the complete array returned by `/v1/resume/embed`.

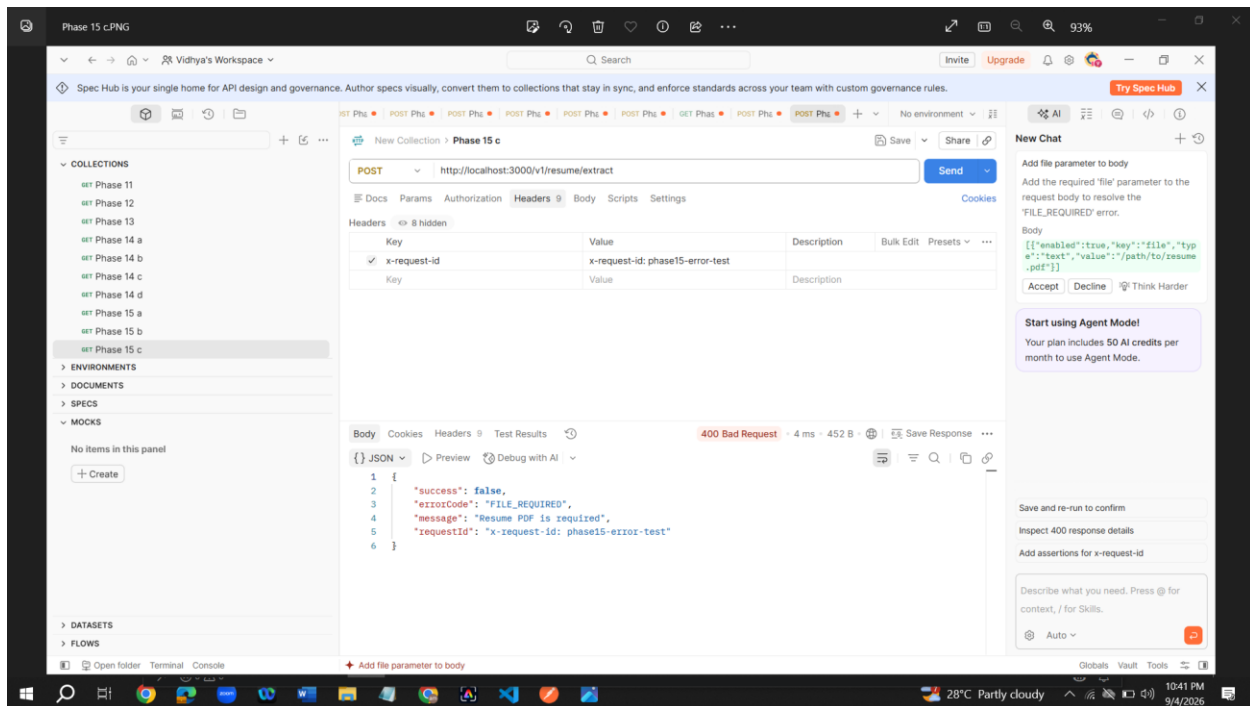
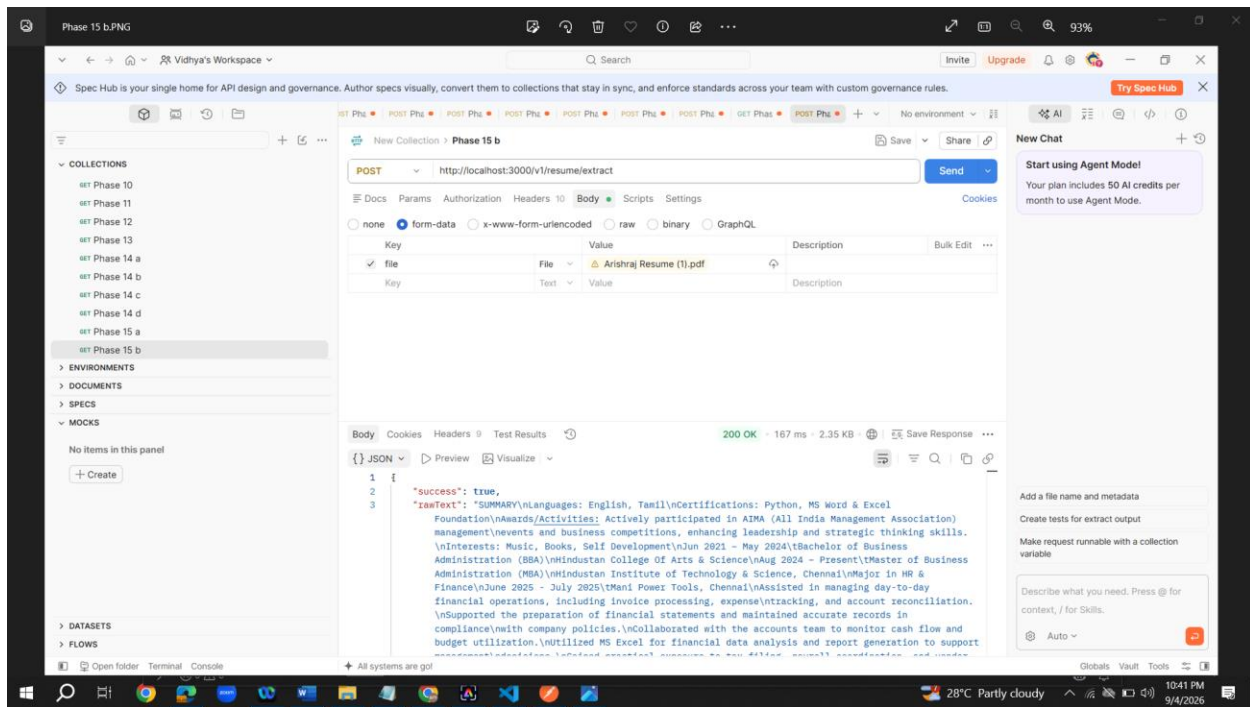
Note : all resumes should be ingested in Mongodb



28. For Phase 14, just copy the contents give by copilet in Postman following the instructions.

29. For Phase 15, just copy the contents give by copilet in Postman following the instructions.

3 steps



30. For Phase 16, just copy the contents give by copilot in Postman following the instructions.

Phase 16 a.PNG

Spec Hub is your single home for API design and governance. Author specs visually, convert them to collections that stay in sync, and enforce standards across your team with custom governance rules.

Invited Upgrade

Try Spec Hub

COLLECTIONS

- Phase 12
- Phase 13
- Phase 14 a
- Phase 14 b
- Phase 14 c
- Phase 14 d
- Phase 15 a
- Phase 15 b
- Phase 15 c
- Phase 16 a

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

No items in this panel

+ Create

DATASETS

Flows

New Collection > Phase 16 a

GET http://localhost:3000/v1/health

Send

Docs Params Authorization Headers Body Scripts Settings

Headers 7 hidden

Key	Value	Description	Bulk Edit	Presets
x-request-id	x-request-id: phase16-health			
Key	Value	Description		

Body Cookies Headers 9 Test Results

200 OK · 74 ms · 386 B

Save Response

JSON Preview Visualize

```
1 {
2   "status": "ok",
3   "app": "resume-tag-backend",
4   "version": "1.0.0",
5   "uptime": 929.6
6 }
```

Finish request headers & URL

Add test for health response

Parameterize x-request-id

Describe what you need. Press @ for context, / for Skills.

Auto

Open folder Terminal Console

All systems are go!

28°C Partly cloudy 10:43 PM 9/4/2026

Phase 16 b.PNG

Spec Hub is your single home for API design and governance. Author specs visually, convert them to collections that stay in sync, and enforce standards across your team with custom governance rules.

Invited Upgrade

Try Spec Hub

COLLECTIONS

- Phase 13
- Phase 14 a
- Phase 14 b
- Phase 14 c
- Phase 14 d
- Phase 15 a
- Phase 15 b
- Phase 15 c
- Phase 16 a
- Phase 16 b

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

No items in this panel

+ Create

DATASETS

Flows

New Collection > Phase 16 b

GET http://localhost:3000/v1/health/db

Send

Docs Params Authorization Headers Body Scripts Settings

Headers 7 hidden

Key	Value	Description	Bulk Edit	Presets
x-request-id	x-request-id: phase16-db-health			
Key	Value	Description		

Body Cookies Headers 9 Test Results

200 OK · 81 ms · 382 B

Save Response

JSON Preview Visualize

```
1 {
2   "status": "ok",
3   "database": "mongodb",
4   "connected": true,
5   "latencyMs": 58
6 }
```

Add negative-case header tests

Parameterize URL and header

Promote common headers to collection variables

Describe what you need. Press @ for context, / for Skills.

Auto

Open folder Terminal Console

All systems are go!

28°C Partly cloudy 10:43 PM 9/4/2026

